CS3510 – Compiler Construction

Semester Project Report

Course	CS3510 – Compiler Construction
Project	Option A — Parsing Phase
Parser	LR(0) and SLR(1) Bottom-Up Parser
Platform	Web Application — HTML / CSS / JavaScript
Submitted	Microsoft Teams
GitHub	https://github.com/HishmatMalhani/Compiler-construction-LR-0-and-SLR-1-

1. Problem Description

Parsing is the second major phase of a compiler. After the lexer breaks source code into tokens, the parser checks whether those tokens form a valid program according to the grammar of the language. For this project we implemented a bottom-up LR parser as an interactive web application.

The tool accepts any context-free grammar (CFG), builds the canonical LR(0) item sets, constructs the SLR(1) parse table, and then parses an input string step by step — showing the exact sequence of shifts and reduces the parser takes, and drawing the parse tree for strings that are accepted.

We chose LR parsing because it handles a wider class of grammars than top-down parsers. Left-recursive grammars like E → E + T work fine with LR but break LL parsers entirely. LR parsers also form the basis of real-world compilers — GCC and Clang both use variants of this approach.

---

2. Selected Compiler Phase

Phase: Parsing (Syntax Analysis). We implemented both LR(0) and SLR(1), with a toggle in the UI to switch between them on the same grammar.

Property	LR(0)	SLR(1)	Notes
Lookahead	None	FOLLOW set (1 symbol)	
Conflicts	More common	Fewer — FOLLOW restricts reduces	
Grammar class	Strict LR(0)	Superset of LR(0)	
Item sets	Same construction	Same construction	No rebuild needed
Table reduces	All terminals	Only FOLLOW(A)	Key difference

The main practical difference: LR(0) reduces in every column whenever a state contains a complete item, which causes conflicts on most real grammars. SLR(1) restricts reduces to tokens that can actually follow the non-terminal — which is usually enough to resolve the conflicts without the full complexity of LALR(1) or canonical LR(1).

---

3. Design and Implementation Overview

3.1 Architecture

Single-page web application built with vanilla HTML, CSS, and JavaScript. No frameworks, no build tools, no external dependencies. Everything runs client-side in the browser. The codebase is split into eight modules inside the one file:

- Grammar Parser — reads CFG text, validates syntax, extracts productions/terminals/non-terminals/start symbol.
- Augmentor — adds S' → S for LR construction.
- FIRST / FOLLOW — fixed-point algorithms iterated until stable; handles epsilon chains.
- LR(0) Builder — Closure and GOTO functions; canonical collection of LR(0) item sets.
- Table Builder — fills ACTION/GOTO; records both entries on conflict rather than overwriting.
- Parse Engine — stack-based shift/reduce loop; produces the step trace.
- Tree Renderer — canvas-based recursive draw; horizontal spacing proportional to leaf count so subtrees never overlap.
- GUI Layer — six tabs, live status bar, three input modes, conflict banners.

3.2 Key Algorithms

Closure(I): For every item [A → α • B β], add all items [B → • γ] for each B-production. Repeat until fixed point. Epsilon productions require special handling — the dot cannot advance over ε, and FIRST must propagate nullability through chains.

GOTO(I, X): Move the dot past symbol X in every applicable item, then run Closure on the result. The canonical collection is built by starting from Closure([S' → • S]) and applying GOTO for every grammar symbol until no new states appear.

SLR(1) Table Fill: Shift entries come from GOTO on terminals. Reduce [A → α •] is placed in ACTION[i, a] for all a ∈ FOLLOW(A). Accept goes at [S' → S •, $]. Conflict = cell already occupied — both entries are stored and the cell is flagged.

3.3 GUI Features

- Three input modes: manual text, .txt file upload, and four built-in example grammars.
- LR(0) / SLR(1) toggle switch — switch on the same grammar and watch conflicts appear or resolve.
- Six tabs: Grammar · LR(0) Items · FIRST/FOLLOW · Parse Table · Parse Steps · Parse Tree.
- Colour-coded parse table: shift = blue, reduce = red, accept = green, GOTO = purple.
- Conflict cells highlighted in red with a warning banner listing each conflict.
- Step-by-step trace table: stack state numbers, remaining input, action at each step.
- Canvas-rendered parse tree: blue nodes for non-terminals, green leaves for terminals.

---

4. Sample Inputs and Manual Working

4.1 Example Grammar — Arithmetic Expressions

```
E → E + T | T    T → T * F | F    F → ( E ) | id
```

After augmentation: E' → E. SLR(1) construction produces 12 states, no conflicts. The grammar encodes operator precedence through structure — multiplication binds tighter than addition because T is defined below E in the production hierarchy.

4.2 Parse Trace — Input: id + id * id

Step	Stack	Input Remaining	Action
1	0	id + id * id $	Shift → s5
2	0 5	+ id * id $	Reduce r6: F → id
3	0 3	+ id * id $	Reduce r4: T → F
4	0 2	+ id * id $	Reduce r2: E → T
5	0 1	+ id * id $	Shift → s6
6	0 1 6	id * id $	Shift → s5
7	0 1 6 5	* id $	Reduce r6: F → id
8	0 1 6 3	* id $	Shift → s7
9	0 1 6 3 7	id $	Shift → s5
10	0 1 6 3 7 5	$	Reduce r6: F → id
11	0 1 6 3 7 10	$	Reduce r5: T → T*F
12	0 1 6 9	$	Reduce r1: E → E+T
13	0 1	$	ACCEPT ✓

Steps 6–11 handle the id * id sub-expression entirely before the + reduction fires in step 12. This is operator precedence enforced purely by grammar structure — the parser applies no special rules.

---

5. Results and Observations

5.1 Test Cases

#	Grammar	Input	Parser	Result
1	Arithmetic Expr	id + id * id	SLR(1)	ACCEPTED ✓
2	Arithmetic Expr	(id + id)	SLR(1)	ACCEPTED ✓
3	Arithmetic Expr	id + * id	SLR(1)	REJECTED ✗
4	Simple S→aAb	a c b	LR(0)	ACCEPTED ✓
5	Simple S→aAb	a a b	LR(0)	REJECTED ✗
6	Balanced Parens	(( ))	SLR(1)	ACCEPTED ✓

5.2 GUI Tab Descriptions

Grammar Tab: Augmented grammar with production numbers. Terminals and non-terminals shown as labelled pills. Start symbol marked.

LR(0) Items Tab: Each item set rendered as a card. Dot position highlighted inside each item. Outgoing GOTO transitions listed below each state.

FIRST / FOLLOW Tab: Side-by-side grid of FIRST and FOLLOW sets for every non-terminal.

Parse Table Tab: Full colour-coded ACTION/GOTO table. Shift entries blue, reduces red, accept green, GOTO purple. Conflict cells are red with a warning banner above the table listing each conflict.

Parse Steps Tab: Numbered trace: stack state numbers, remaining input tokens, action taken. Final row turns green on ACCEPT or red on REJECT.

Parse Tree Tab: Canvas-drawn tree for accepted strings. Internal nodes (non-terminals) are blue circles; leaf nodes (terminals) are green. Horizontal spacing is proportional to leaf count so no two subtrees overlap.

6. Challenges and Observations

Epsilon Productions: Every stage needed special handling for ε. Closure can't advance the dot over it. FIRST must propagate nullability through chains — a bug here caused wrong FOLLOW sets on grammars with multiple nullable non-terminals, and the symptom showed up in the parse table rather than where the mistake actually was.

Conflict Detection: We chose to record both entries on a conflict rather than silently overwriting. The table builder stores a list per cell and the renderer flags conflicts in red. This turned out to be the most useful feature for understanding what a conflict actually means — seeing 's5' and 'r3' in the same cell makes it concrete.

LR(0) vs. SLR(1) Toggle: Sharing item sets between both modes while having different table-fill logic required clean separation. Item construction runs once; the table fill is a separate pass that takes the mode as a parameter. The toggle became one of the most effective teaching features — students can load a conflicting grammar, flip the switch, and watch the red cells disappear.

Parse Tree Layout: Automatic layout without overlap is harder than it looks. Trees with uneven branching depth would collide. The fix was assigning each node horizontal space proportional to the number of terminal leaves beneath it — this guarantees no two subtrees ever share horizontal space regardless of grammar depth.

No External Libraries: Keeping the tool dependency-free means it runs offline, loads instantly, and submits as a single file. This simplified testing and means the grader needs only a browser to run it.

All six test cases produce the expected results. The implementation covers every mandatory requirement for Option A: GUI with manual and file input, grammar validation, step-by-step parsing with accept/reject status, and parse tree generation for valid strings.